

# Mitigating Forgetting in LLM Fine-Tuning via Low-Perplexity Token Learning

Chao-Chung Wu<sup>1\*</sup> Zhi Rui Tam<sup>1\*</sup> Chieh-Yen Lin<sup>1</sup> Yun-Nung Chen<sup>2</sup>  
 Shao-Hua Sun<sup>1,2†‡</sup> Hung-yi Lee<sup>2†</sup>  
<sup>1</sup>Appier AI Research, <sup>2</sup>National Taiwan University  
 {johnson.wu, ray.tam}@appier.com

## Abstract

Maintaining consistent model performance across domains is a fundamental challenge in machine learning. While recent work has explored using LLM-generated data for fine-tuning, its impact on cross-domain generalization remains poorly understood. This paper presents a systematic analysis revealing that fine-tuning with LLM-generated data not only improves target task performance but also reduces non-target task degradation compared to fine-tuning with ground truth data. Through analyzing the data sequence in tasks of various domains, we demonstrate that this enhancement of non-target task robustness stems from the reduction of high perplexity tokens found in LLM-generated sequences. Following our findings, we showed that masking high perplexity tokens in ground truth training data achieves similar non-target task performance preservation, comparable to using LLM-generated data. Extensive experiments across different model families and scales, including Gemma 2 IT 2B, Llama 3 8B Instruct, and three additional models, agree with our findings. To the best of our knowledge, this is the first work to provide an empirical explanation based on token perplexity reduction to mitigate catastrophic forgetting in LLMs after fine-tuning, offering valuable insights for developing more robust fine-tuning strategies.

## 1 Introduction

Supervised fine-tuning large language models (LLMs) [29] has proven highly effective for enhancing their ability to follow novel instructions [15, 21, 23, 43, 49] and produce useful outputs across a wide range of tasks, including summarization and web querying [1, 17, 26, 34]. As LLMs are increasingly applied in specialized domains such as arithmetic and programming assistance [27, 38], practitioners often encounter substantial computational and data cleaning issues when adapting these models to gain improvements. These difficulties are exacerbated by performance saturation in smaller LLMs, where benchmark improvements on tasks like MATH [12] and MBPP [3] tend to plateau as model size decreases. Moreover, fine-tuning can risk degrading a model’s general capabilities [4, 24, 30, 31]. This raises a critical question: how can we efficiently fine-tune pre-trained, instruction-following LLMs for domain-specific tasks *without compromising other skill sets such as arithmetic or common sense reasoning*, especially when the original fine-tuning data is unavailable or computational resources are constrained?

Recent work has shown that fine-tuning with model-generated data can be highly effective, often outperforming training on ground truth data when using high-quality generated samples [10, 32,

\*Equal contribution

†Equal advisory contribution

‡Shao-Hua Sun is an Adjunct Research Scientist at Appier AI Research

Our code and datasets are available at <https://github.com/appier-research/robust-llm-finetunes>

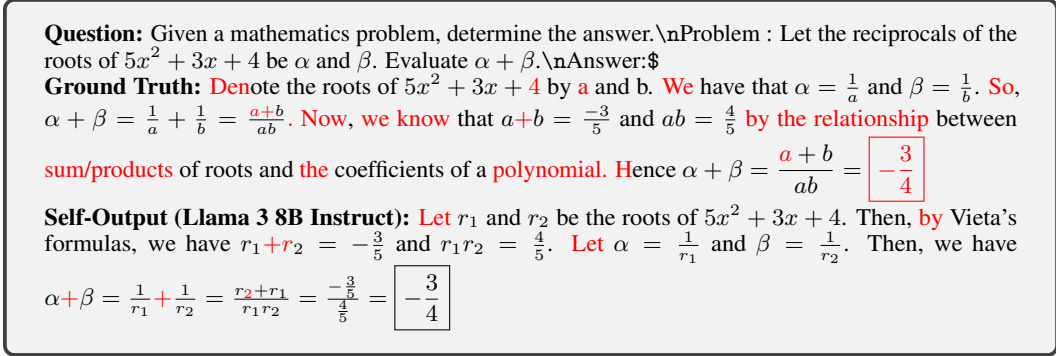


Figure 1: An example MATH problem showing more high perplexity tokens (highlighted in red, perplexity  $\geq 2.5$ ) in ground truth than Self-Output responses (Llama 3 self-generated responses).

35, 48]. For example, Ren et al. [31] demonstrated that model-generated responses can surpass human annotations in downstream performance, particularly when using more capable models (*e.g.*, GPT-4) to generate training data and mitigate degradation on general tasks. Although they attributed part of this success to the “familiarity” of generated responses, measured via low perplexity, it remains unclear why such data works well for instruction tuning. Specifically: (1) The regenerated response may differ contextually from the ground truth, and such contextual bias could affect the utility of low-perplexity training. (2) The “non-forgetting” impact of low-perplexity training on the performance of tasks not being fine-tuned, which maintains the practicality of the model, is still underexplored. (3) Since model size plays a role in distillation benefits, it is important to disentangle the effects of model scale and evaluate low-perplexity training using models of the same size.

Our work offers an empirical explanation, grounded in loss and perplexity reduction, for why LLM-generated data tends to cause less degradation in fine-tuning outcomes. Through systematic analysis, we first show that fine-tuning with different types of LLM-generated data (*e.g.*, self-generated and rephrased responses) significantly mitigates degradation on non-target tasks compared to fine-tuning with ground truth data. Second, we trace this improvement to a key property illustrated in Figure 1: LLM-generated responses exhibit lower sentence-level perplexity and a smaller proportion of high-perplexity tokens. This leads to a practical insight: by simply masking high-perplexity tokens in ground truth training data, we can achieve a comparable reduction in non-target task degradation as observed with LLM-generated data. Importantly, this can be done using the same model prior to training, allowing us to pre-process the data and harness the benefits of low-perplexity training.

We propose a novel empirical explanation for non-target task performance degradation, introducing Selective Token Masking (STM), a simple yet effective strategy that filters out high-perplexity tokens to enable low-perplexity training. We have applied STM on (1) different perplexity criteria, (2) different model families and scales, and (3) various fine-tuning techniques from full-weight fine-tuning to parameter-efficient fine-tuning. We found that STM could achieve almost the same capabilities as the original models have on non-target tasks after fine-tuning on target tasks under all circumstances. We could identify that the degradation results from the low model weights update during the low-perplexity training. Together, these insights provide a new perspective on designing fine-tuning strategies that better retain general capabilities while enhancing performance on domain-specific tasks.

## 2 Training LLMs on self-generated data

To evaluate the effectiveness of LLM-generated data for fine-tuning, we explore two distinct methods for generating training data across two different target tasks. We then assess the models fine-tuned with the generated data on five non-target tasks, respectively. The two generation strategies, Self-Output and Rephrase, offer complementary approaches to constructing LLM-based training data, each addressing different trade-offs and challenges. In this section, we introduce the original datasets used for data generation and describe our methodology for constructing self-generated training datasets using language models.

## 2.1 Training and evaluation framework

For target-task fine-tuning, we adopt the MBPP and MATH datasets. These datasets provide rich annotations, including full solutions, reasoning steps, and test cases, beyond just the final answers, allowing models to learn comprehensive task-solving behaviors. For evaluation, in addition to using the test sets from MBPP and MATH, we assess model performance on GSM8K [7], ARC-Challenge [6], and BIRD [19] to examine generalization to non-target tasks involving various forms of reasoning and generation. A detailed description of the datasets is provided in Section 4.

## 2.2 Self-Output

Self-Output follows a similar high-level approach, similar to other synthetic data generation techniques [31, 48], where a high-quality synthetic dataset is created by sampling diverse responses from an LLM and applying a strict filtering process. For each training instance, we use a language model  $M$  (*e.g.*, Llama 3 8B Instruct) to generate  $N$  (*e.g.*, 32) distinct responses with a temperature setting of  $T = 0.7$ . The generated responses are then filtered to those that are semantically aligned with the ground truth, ensuring the quality and relevance of synthetic data.

Self-Output excels at producing high-quality training data in domains with reliably verifiable outputs (*e.g.*, programming tasks with objective ground truths). However, it remains constrained to settings where such verification is feasible and imposes significant computational overhead from the multiple generations and filtering required to identify valid outputs.

Table 1: Perplexity average and variance over answer sequences of MBPP and MATH training datasets on Llama 3 8B Instruct. The average sentence perplexity is calculated by averaging the sum of each sentence’s perplexity over the token perplexity score.

| Data | Method       | Avg. PPL    |
|------|--------------|-------------|
| MBPP | Ground Truth | 4.83 (7.04) |
|      | Rephrase     | 1.69 (0.16) |
|      | Self-Output  | 1.16 (0.01) |
| MATH | Ground Truth | 2.45 (0.81) |
|      | Rephrase     | 2.11 (9.28) |
|      | Self-Output  | 1.34 (0.03) |

## 2.3 Rephrase

Yang et al. [47] proposed self-distillation. The process uses an instruction-finetuned LLM to rephrase ground-truth responses in its style. By providing both the instruction and ground truth to the target LLM, it generates semantically equivalent reformulations with ground truth responses. Compared to Self-Output, Rephrase is computationally efficient, as it requires only a single pass of generation without additional filtering. However, Rephrase is susceptible to hallucinations, as evidenced by the higher variance in token-level perplexity (see Table 1). Despite this limitation, Rephrase offers versatility, as it does not rely on verifiable ground truth and can be applied to a broader range of tasks.

We have a sanity check on the output of Rephrase, and remove the incorrect label output as the final training set for Rephrase to ensure that all the final outputs of Rephrase are correct. But we did not evaluate the intermediate output of Rephrase, *e.g.*, the reasoning or CoT process. This process is partially reflected in Appendix F, Table 17.

Table 2 presents the performance comparison between Self-Output, Rephrase, and Baseline Fine-tuning on ground truth using different models in terms of model size and model series across both target task improvement and non-target task degradation rate. Both task improvement rate (TI) and degradation rate (BWT) are adopted from Backward Transfer rate from [14] and modified as follows:

$$\mathbf{TI} = (a_{\text{target}}^{(\text{train})} - a_{\text{target}}^{(\text{original})}) / a_{\text{target}}^{(\text{original})}. \quad (1)$$

$$\mathbf{BWT} = \frac{1}{T-1} \sum_{i=1}^{T-1} (a_i^{(\text{train})} - a_i^{(\text{original})}) / a_i^{(\text{original})}. \quad (2)$$

Given  $T$  tasks composed of one target task and  $T-1$  non-target tasks, **TI** is the target task performance improvement percentage after training on the target task. **BWT** refers to the average performance degradation percentage of the  $T-1$  non-target task after training on the target task. The models to fine-tune in this paper refer to generic instruction following models before fine-tuning with target task data, *e.g.*, Llama 3 8B Instruct [8], Gemma 2 IT 2B [36], Mistral 7B Instruct [16], and OLMo 2 7B series [40]. For target task performance, Self-Output demonstrates consistent positive **TI** in both MBPP and MATH. For the non-target task, the Self-Output method achieves better BWT scores